CSCI-473: Applied Data Science
Homework Assignment 2
Exploratory and Statistical Analysis of Bike-Sharing Demand

Weight:	5% of the final course grade
Assignment Type:	Individual
Programming Language:	Python
Due Date:	October 26, 2026, at 11:59 PM EST
Instructor:	Desta Haileselassie Hagos, PhD
Questions:	desta.hagos@howard.edu

Important

Read the complete assignment instructions before beginning your work. Submit exactly **one Python source file** and **one PDF report** through Canvas. This is an individual assignment.

1 Purpose of the Assignment

This homework gives you practice applying exploratory data analysis and statistical reasoning to a real dataset. You will use pandas, NumPy, Matplotlib, and Seaborn to prepare, validate, summarize, visualize, and interpret hourly bike-sharing data. The assignment focuses on concepts covered in Lectures 1–7, including:

- loading and inspecting CSV data;
- working with paths and exporting results;
- converting dates and coded categories;
- validating values with Boolean conditions and assertions;
- filtering, grouping, aggregating, and sorting data;
- conducting univariate, bivariate, and multivariate EDA;
- creating clear and appropriate visualizations;
- calculating and interpreting Pearson correlations;
- estimating empirical and conditional probabilities;
- examining sampling variability; and
- constructing a bootstrap confidence interval.

You are not required to build a machine-learning model or conduct a hypothesis test in this assignment.

2 Dataset

You will work with the **UCI Bike Sharing dataset**. The dataset was created by Hadi Fanaee-T and contains hourly bicycle-rental records from the Capital Bikeshare system for 2011 and 2012. The provided file contains **17,379 hourly observations**. It includes:

- the date and hour of each observation;
- coded categories for season, weather, month, and weekday;
- indicators for holidays and working days;
- normalized temperature, apparent temperature, humidity, and wind speed measurements; and
- counts of casual, registered, and total rentals.

The dataset is distributed under a CC BY 4.0 license. Additional information is available from the UCI Machine Learning Repository:

`https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset`

Dataset Files

Download `bike_sharing_hour.csv` and `Bike_Sharing_Dataset_README.txt` from the Homework 2 assignment page on Canvas. Save both files in the same folder as your Python file. Use `bike_sharing_hour.csv` for all programming and analysis tasks. The README file is provided only as a reference for understanding the variables and category codes. Do not use `day.csv` for this assignment. Use the files provided on Canvas so that everyone works with the same version of the dataset.

Variables Used in This Assignment

Column	Meaning
`instant`	Record identifier
`dteday`	Calendar date
`season`	Season code: 1 spring, 2 summer, 3 fall, 4 winter
`yr`	Year code: 0 for 2011 and 1 for 2012
`mnth`	Month number from 1 to 12
`hr`	Hour of the day from 0 to 23
`holiday`	Holiday indicator
`weekday`	Day-of-week code
`workingday`	1 if the day is neither a weekend nor a holiday; otherwise 0
`weathersit`	Weather-condition code from 1 to 4
`temp`	Normalized temperature
`atemp`	Normalized apparent temperature
`hum`	Normalized humidity
`windspeed`	Normalized wind speed
`casual`	Number of rentals by casual users
`registered`	Number of rentals by registered users
`cnt`	Total number of rentals

3 General Programming Requirements

Your program must:

- use pandas, NumPy, Matplotlib, and Seaborn;
- complete all six tasks in the order presented;
- contain clear comments identifying each task;
- use descriptive variable names;
- run from beginning to end without errors;
- print clearly labeled results for every task;
- set a random seed whenever random sampling is performed;
- create the `output` folder programmatically if it does not already exist;
- save every required figure in an `output` folder; and
- save the prepared dataset as `bike_sharing_prepared.csv`.

Do not manually calculate results that Python can produce. Your reported answers, figures, and numerical results must be generated by your program.

4 Task 1: Load and Audit the Dataset

Load the provided CSV file into a pandas DataFrame named `df`.

Your program must display:

1. the first five rows;
2. the last five rows;
3. the number of rows and columns;
4. all column names;
5. the data type of every column;
6. the output of `info()`;
7. descriptive statistics for the numerical columns;
8. the number of missing values in each column; and
9. the number of exact duplicate rows.

After inspecting the output, identify at least **three issues or features that require attention before analysis**. These do not need to be errors. Examples include coded categories, normalized

measurements, a date stored as text, or columns whose mathematical relationship affects later interpretation.

Code Hint

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("bike_sharing_hour.csv")

print("Shape:", df.shape)
print("\nMissing values:")
print(df.isna().sum())
print("\nExact duplicates:", df.duplicated().sum())
```

Report Requirement

Explain what the audit revealed. Include at least one readable screenshot showing the structural inspection, missing-value counts, and duplicate count.

5 Task 2: Prepare the Variables for Analysis

Create a working copy of the original DataFrame before making changes:

```
raw_df = df.copy()
clean_df = df.copy()
```

Rename the following columns so that their meanings are clearer:

Original Name	Required Clean Name
dteday	date
yr	year_code
mnth	month
hr	hour
weathersit	weather_code
temp	temperature_normalized
atemp	apparent_temperature_normalized
hum	humidity_normalized
windspeed	wind_speed_normalized
casual	casual_rentals
registered	registered_rentals
cnt	total_rentals

Then complete the following preparation steps:

1. convert `date` to a pandas date-time data type using `pd.to_datetime()` with `errors="coerce"`;
2. create a `year` column containing 2011 or 2012;

3. create a `season_label` column containing `Spring`, `Summer`, `Fall`, or `Winter`;
4. create a `weather_label` column using the following labels:
   `Clear/Partly Cloudy`, `Mist/Cloudy`,
   `Light Rain/Snow`, and `Heavy Rain/Snow/Fog` for weather codes 1, 2, 3, and 4, respectively;
5. create a `day_type` column that maps `workingday`=1 to `Working Day` and `workingday`=0 to `Nonworking Day`;
6. create `temperature_c`, `apparent_temperature_c`, `humidity_percent`, and `wind_speed` using the conversion formulas below; and
7. print the resulting column names and data types, followed by a small sample of the prepared rows, such as the first five rows, showing the newly created columns.

Use the following documented conversions:

$$\texttt{temperature_c} = 47 \times \texttt{temperature_normalized} - 8$$

$$\texttt{apparent_temperature_c} = 66 \times \texttt{apparent_temperature_normalized} - 16$$

$$\texttt{humidity_percent} = 100 \times \texttt{humidity_normalized}$$

$$\texttt{wind_speed} = 67 \times \texttt{wind_speed_normalized}$$

Code Hint

```
from pathlib import Path

output_folder = Path("output")
output_folder.mkdir(
    parents=True,
    exist_ok=True)
```

```
season_map = {
    1: "Spring",
    2: "Summer",
    3: "Fall",
    4: "Winter"
}

clean_df["season_label"] = (
    clean_df["season"].map(season_map))

clean_df["temperature_c"] = (
    47 * clean_df["temperature_normalized"] - 8)
```

Report Requirement

Describe the renaming, conversion, and category-labeling steps. Include a screenshot showing the new columns, resulting data types, and several prepared rows.

6 Task 3: Validate the Prepared Dataset

Perform the following validation checks:

1. verify that `instant` contains unique values;
2. verify that `hour` is between 0 and 23;
3. verify that `month` is between 1 and 12;
4. verify that `season` and `weather_code` contain only their documented codes;
5. verify that `holiday` and `workingday` contain only 0 and 1;
6. verify that all rental-count columns contain nonnegative values;
7. verify that `total_rentals = casual_rentals + registered_rentals`;
8. verify that `date`, `hour`, `season_label`, `weather_label`, `temperature_c`, `humidity_percent`, `wind_speed`, and `total_rentals` contain no missing values; and
9. verify that the prepared dataset contains no exact duplicate rows.

Use Boolean conditions and at least **five assertions**. Print a clearly labeled confirmation message after all assertions pass.

Code Hint

```
assert clean_df["instant"].is_unique
assert clean_df["hour"].between(0, 23).all()
assert set(clean_df["workingday"].unique()).issubset({0, 1})

assert (
    clean_df["total_rentals"]
    == clean_df["casual_rentals"]
       + clean_df["registered_rentals"]
).all()

assert clean_df.duplicated().sum() == 0
```

Important Relationship

The total rental count is defined as `casual_rentals + registered_rentals`. Therefore, strong correlations between these component columns and `total_rentals` should not be presented as discoveries. The relationship is built into the definition of the total.

Report Requirement

List the rules you validated and explain what each assertion checks. Include a clear screenshot showing the successful validation output.

7 Task 4: Exploratory Data Analysis and Visualization

Use the prepared DataFrame to complete all analyses below.

A. Numerical and categorical summaries

Calculate and display:

1. the mean, median, standard deviation, minimum, maximum, and quartiles of `total_rentals`;
2. the number of observations in each season and weather category;
3. the mean and median total rentals for working and nonworking days;
4. the mean total rentals for every hour of the day; and
5. the number and mean total rentals grouped by both `season_label` and `workingday`.

Use `groupby()` and `agg()` where multiple statistics are required. Round displayed numerical results to two decimal places.

B. Required visualizations

Create and save all four visualizations:

1. a histogram of `total_rentals`;
2. a box plot of total rentals by season;
3. a line plot of mean total rentals by hour, with separate lines for working and nonworking days; and
4. a scatter plot of temperature in degrees Celsius versus total rentals, with color representing season.

Every figure must have a descriptive title, clear axis labels, and an appropriate legend when needed. Use `plt.tight_layout()`, save each figure in the `output` folder, and close each completed figure.

Code Hint

```
hourly_summary = clean_df.groupby(
    ["hour", "day_type"]
)["total_rentals"].mean().reset_index()

sns.lineplot(data=hourly_summary,
    x="hour",
    y="total_rentals",
    hue="day_type")
```

```
plt.xlabel("Hour of Day")
plt.ylabel("Mean Total Rentals")
plt.tight_layout()
plt.savefig("output/hourly_demand.png", dpi=300)
plt.close()
```

Report Requirement

Include all four figures. For each figure, write two or three sentences describing the main pattern and any important limitation. Do not merely repeat the axis labels.

8 Task 5: Probability, Correlation, and Interpretation

A. Empirical and conditional probability

Define a high-demand hour as an observation whose `total_rentals` value is greater than or equal to the dataset's 75th percentile. Create a Boolean column named `high_demand`.

Calculate and report:

1. the 75th-percentile threshold;
2. $P(\text{high demand})$;
3. $P(\text{high demand} \mid \text{working day})$;
4. $P(\text{high demand} \mid \text{nonworking day})$; and
5. $P(\text{high demand} \mid \text{clear or partly cloudy weather})$, using weather code 1 for the condition.

Show the numerator and denominator used for every conditional probability. Interpret the results in context.

B. Pearson correlation

Create a Pearson correlation matrix using:

- `temperature_c`,
- `apparent_temperature_c`,
- `humidity_percent`,
- `wind_speed`, and
- `total_rentals`.

Display the matrix rounded to two decimal places and create an annotated heatmap. Identify the weather variable with the strongest absolute Pearson correlation with total rentals. Report its correlation coefficient and interpret its direction and strength carefully.

Code Hint

```
high_demand_threshold = clean_df[
    "total_rentals"
].quantile(0.75)

clean_df["high_demand"] = (
    clean_df["total_rentals"]
    >= high_demand_threshold)

working_rows = clean_df["workingday"] == 1

p_high_given_working = (
    (clean_df["high_demand"] & working_rows).sum()
    / working_rows.sum())
```

Percentile Threshold

Because multiple observations may equal the 75th-percentile threshold, the empirical probability of `high_demand` is not guaranteed to be exactly 0.25. Calculate it from the Boolean column rather than assuming its value.

Remember to Interpret Correlation Carefully

Pearson correlation summarizes linear association. It does not establish causation. Examine the scatter plot from Task 4 together with the correlation coefficient, and discuss possible seasonal, temporal, or behavioral factors that may affect the relationship.

Report Requirement

Report and interpret all probabilities. Include the correlation matrix and heatmap, identify the strongest weather-variable correlation, and explain why the result should not be interpreted as causal.

9 Task 6: Sampling, Bootstrap Uncertainty, and Export

A. One reproducible random sample

Select a simple random sample of **500 rows without replacement** using `random_state=42`. Calculate the mean total rentals for:

- the complete dataset; and
- the 500-row sample.

Report the difference and explain why the two means need not be identical.

B. Repeated sampling

Draw **200 random samples without replacement**, each containing 500 rows. Use a different reproducible sample during each repetition. Store the mean total rentals from every sample. Then:

1. report the mean of the 200 sample means;
2. report the standard deviation of the sample means; and
3. create a histogram of the sample means with a vertical line showing the full-dataset mean.

Use a reproducible random-number generator. Do not reset the random-number generator inside the loop.

C. Bootstrap confidence interval

Using the observed `total_rentals` values:

1. generate **5,000 bootstrap samples** of the same size as the original dataset, sampling with replacement;
2. calculate the mean for each bootstrap sample;
3. estimate the bootstrap standard error; and
4. use the 2.5th and 97.5th percentiles to construct an approximate 95% bootstrap confidence interval for mean hourly rentals.

Interpret the confidence interval correctly. Do not describe it as the range containing 95% of individual hourly rental counts.

Important Limitation

The dataset contains time-ordered hourly observations from one bike-sharing system during 2011 and 2012. Neighboring observations may not be independent. The simple row bootstrap used in this assignment is an instructional approximation and does not, by itself, justify generalizing the interval to other cities, systems, or time periods. Discuss this limitation in your report.

D. Export and verify

Save the prepared DataFrame as:

`bike_sharing_prepared.csv`

Do not save the DataFrame index as a separate column. Reload the saved file into a new DataFrame and display:

- its first five rows;
- its shape; and
- its missing-value counts.

Verify that the reloaded shape matches the shape before saving.

Code Hint

```
rng = np.random.default_rng(42)
rentals = clean_df["total_rentals"].to_numpy()

bootstrap_means = []

for _ in range(5000):
    bootstrap_sample = rng.choice(
        rentals,
        size=len(rentals),
        replace=True
    )
    bootstrap_means.append(bootstrap_sample.mean())

lower, upper = np.percentile(
    bootstrap_means,
    [2.5, 97.5]
)
```

Report Requirement

Compare the full-data and sample means, interpret the sampling-distribution histogram, report the bootstrap standard error and confidence interval, and describe how you verified the exported file. Include readable screenshots and the required figure.

10 PDF Report Requirements

Submit one clearly organized PDF report. The report must contain the following sections.

1. Student Information

Include:

- full name (including your middle name);
- student ID number;
- Howard University email address;
- course: CSCI-473; and
- assignment title.

2. Introduction

Briefly describe the dataset, the main analysis question, and the purpose of the assignment.

3. Tasks 1–6

Create a separate, clearly labeled subsection for every task. For each task:

- describe what you did;
- identify the important Python or pandas operations you used;
- report and interpret the results;
- include the required visualizations or output screenshots; and
- explain important assumptions, limitations, or validation decisions.

Screenshots must show the output produced by your submitted Python file. Screenshots of code alone do not satisfy this requirement. Crop screenshots so that the relevant output is readable.

4. Analysis Summary

Summarize:

- the most important demand patterns you found;
- the probability and correlation results;
- what the sampling and bootstrap results showed about uncertainty;
- the limitations of the analysis; and
- how you verified the prepared dataset.

5. GenAI Use Statement

Include the statement described in the Responsible Use of Generative AI section of this document.

6. Conclusion

Briefly explain what you learned about exploratory data analysis, probability, sampling variability, and statistical interpretation.

Code in the Report

Do not paste your entire Python program into the PDF. The complete source code will be submitted separately. You may include short code excerpts when they help explain an important analytical or validation decision.

11 Submission Requirements

Upload exactly the following two files directly to Canvas:

1. `Firstname_Lastname_HW2.py`
2. `Firstname_Lastname_HW2_Report.pdf`

The Python file must contain the complete solution for Tasks 1–6 and must run successfully on its own when the provided CSV file is in the same folder.

The prepared CSV file and saved figures are outputs of your program and do **not** need to be uploaded unless the instructor requests them separately.

How to Submit

Upload the Python source file and PDF report directly to Canvas. Do not combine the files into a ZIP folder. **Submissions sent by email will not be accepted or graded**.

Before submitting, verify that:

- both filenames follow the required naming convention;
- the Python program runs from beginning to end without errors;
- all printed results are clearly labeled;
- every task is explained in the PDF report;
- all figures and screenshots are readable; and
- the GenAI Use Statement is included.

12 Responsible Use of Generative AI

Responsible use of Generative AI tools is permitted for this assignment. Examples of appropriate uses include:

- asking for clarification of a pandas, probability, or statistics concept;
- asking why an error occurred;
- receiving feedback on an analysis or visualization approach;
- generating additional practice examples; and
- checking whether your explanation is clear.

You are responsible for the correctness of every line of code, every result, and every explanation that you submit.

Important

You must understand and be able to explain all submitted work. Do not submit AI-generated code or explanations that you have not independently checked. Do not use methods or libraries that have not been covered in class merely because a Generative AI tool suggested them.

If you used a Generative AI tool, your PDF report must identify:

- the tool you used;
- what you used it for;
- one or more representative prompts or questions you submitted;
- how you checked the accuracy of the response; and
- what you changed or corrected, when applicable.

If you did not use a Generative AI tool, state:

> I did not use a Generative AI tool while completing this assignment.

13 Similarity Review

Submitted reports may be evaluated using **Turnitin**. The similarity score identifies text that matches existing sources; it does not, by itself, establish plagiarism. A similarity score above 20% may trigger additional review, but no grade or academic-integrity decision will be based solely on the numerical score.

Similarity resulting from assignment instructions, required variable names, provided code fragments, properly cited sources, or automatically generated program output will not normally be treated as evidence of misconduct. However, unattributed copying, unauthorized collaboration, or submission of another person's writing or code may constitute an academic-integrity violation.

14 Academic Integrity

This is an individual assignment. You may discuss general course concepts with classmates, but the submitted code, analysis, explanations, figures, and report must be your own work.

You may not:

- copy another student's code, report, screenshots, figures, or results;
- share your completed solution with another student;
- divide the tasks among students;
- copy a complete solution from an online repository;
- fabricate output, figures, or screenshots; or
- submit AI-generated work that you do not understand.

You should be prepared to explain any part of your submitted program or report if asked by the instructor.

15 Grading

Homework Assignment 2 is worth 5% of the final grade of the course and will be graded out of 5 points.

Component	**Points**
Task 1: Loading and initial audit	0.50
Task 2: Variable preparation and conversion	0.50
Task 3: Validation and assertions	0.50
Task 4: EDA, summaries, visualizations, and interpretation	1.00
Task 5: Probability, correlation, and interpretation	1.00
Task 6: Sampling, bootstrap interval, export, and verification	1.00
PDF report, screenshots, figures, GenAI statement, and clarity	0.50
Total	**5.00**

Partial credit may be awarded when the submitted work demonstrates a correct approach but contains a limited implementation or interpretation error.

16 Final Checklist

☐ I used the dataset provided through Canvas.

☐ My single Python file completes Tasks 1–6.

☐ My program runs without errors.

☐ My outputs are clearly labeled.

☐ My PDF explains every task and includes all required figures and screenshots.

☐ I included a GenAI Use Statement.

☐ I used the required filenames.

☐ I uploaded one `.py` file and one PDF file to Canvas.